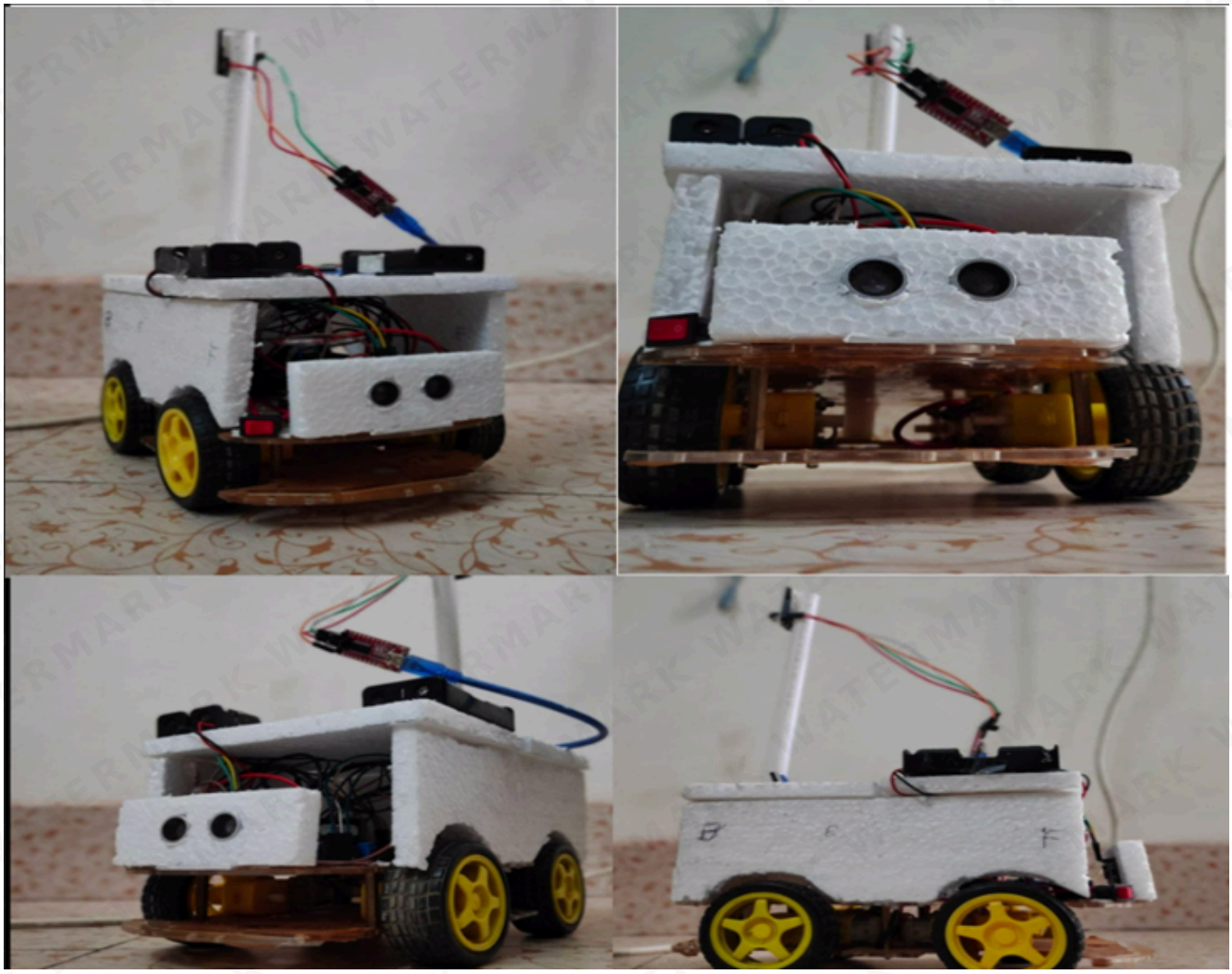
🤖 AI-Based Human Following Robot



PLATFORM ARDUINO | ESP32 | ESP8266 LANGUAGE C++

COMMUNICATION WI-FI STATUS EXPERIMENTAL

📺 Watch Video Tutorial

👥 Team: Robo Inquisitives — Group 01

Name	Student ID
Tahzib Mahmud Rifat	2001005
Md. Numanur Rahman	2001011

Name	Student ID
Md Jalish Mahamud	2001014
Md Moshfiqur Rahaman	2001023
Saiful Islam	2001025

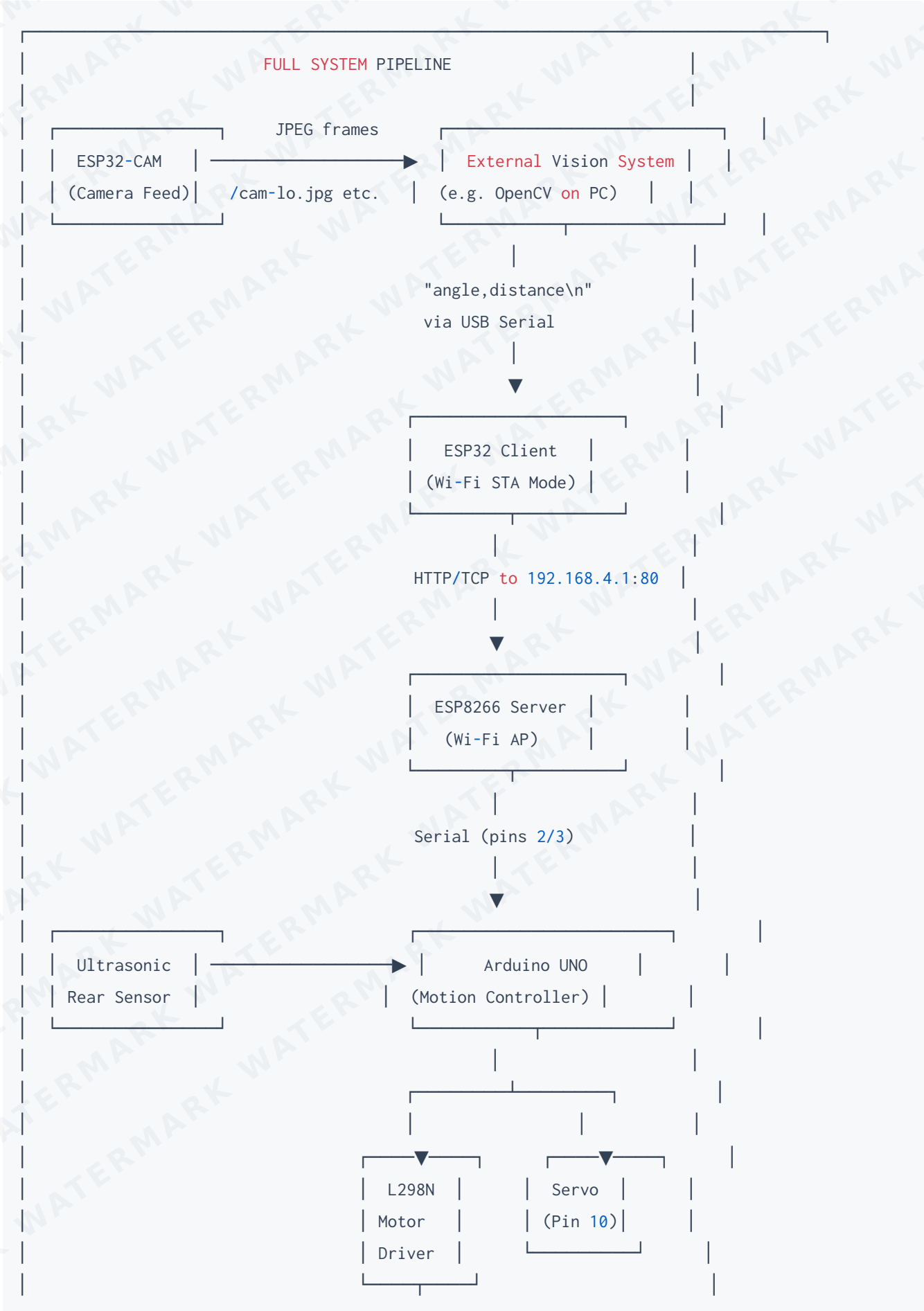
📖 Table of Contents

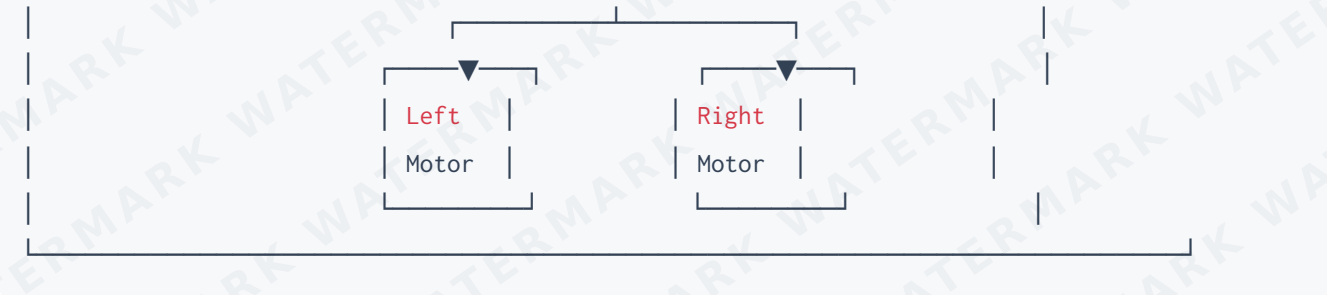
🔍 Overview

This project implements an **AI-assisted human-following robot** built on a distributed embedded systems architecture. A camera module streams live video, which an external vision system (or a human operator) uses to compute the **angle** and **distance** to the target. That data is wirelessly relayed through a chain of microcontrollers, ultimately driving the robot's motors and servo to follow a person autonomously.

> **Key Concept:** The robot does not perform AI inference on-board. Instead, it receives pre-computed `angle,distance` commands over Wi-Fi and executes precise motor/servo responses.

🏗️ System Architecture





⚙️ How It Works — End to End

Step 1 — Camera Streams Live Video (ESP32-CAM)

The **ESP32-CAM** module connects to your local Wi-Fi (Station mode) and hosts an HTTP server that serves JPEG frames at three resolution endpoints:

Endpoint	Quality
`/cam-lo.jpg`	Low resolution
`/cam-mid.jpg`	Medium resolution
`/cam-hi.jpg`	High resolution

An external system (e.g., a laptop running OpenCV) continuously polls these frames to detect a person in the scene.

Step 2 — Vision System Computes Angle & Distance

The external detection module analyzes each frame and produces two values:

Parameter	Meaning	Range
`angle`	Horizontal bearing to the target in degrees	`0-180` (90 = straight ahead)
`distance`	Estimated distance to the target	Integer; ~145–155 is the ideal follow band

These are formatted as a simple ASCII string and written to the ESP32 client over USB Serial:

```
angle,distance\n
```

Example: `90,160\n` — target is directly ahead and a bit far.

Step 3 — ESP32 Client Forwards the Command (Wi-Fi)

The **ESP32 (non-CAM)** operates in Wi-Fi Station mode. It:

1. Reads the `angle,distance` line from its Serial port.
2. Connects to the ESP8266's access point (`SSID: No_Network`).
3. Sends the string as an HTTP/TCP request to `192.168.4.1:80`.

Step 4 — ESP8266 Receives and Relays (Wi-Fi AP → Serial)

The **ESP8266** acts as a Wi-Fi Access Point and TCP server. When data arrives from the ESP32 client:

1. It receives the `angle,distance` string over Wi-Fi.
2. It forwards it over its hardware serial interface to the **Arduino**.

Step 5 — Arduino Parses and Drives the Robot

The **Arduino UNO** is the heart of the motion controller. It:

1. Reads the incoming line from SoftwareSerial (pins 2/3).
2. Parses `angle` and `distance` from the string.
3. Samples the **rear ultrasonic sensor** for obstacles.
4. Computes and executes the appropriate motor + servo commands.

This loop runs continuously as long as Serial data arrives.

🔧 Hardware Requirements

Component	Purpose
Arduino UNO	Main motion controller
ESP8266 Module	Wi-Fi Access Point / serial relay server

Component	Purpose
ESP32 (non-CAM)	Wi-Fi client forwarding angle/distance
ESP32-CAM	Live video stream for vision system
L298N Motor Driver	H-bridge driver for two DC motors
2× DC Drive Motors	Wheel propulsion
Servo Motor	Camera/scan turret, attached to pin 10
HC-SR04 Ultrasonic Sensor	Rear obstacle detection
Battery Pack	Independent motor power supply
Chassis, Wheels, Wiring	Mechanical assembly

> ⚠️ **Important:** Never power the motors from the Arduino's 5V regulator. Use a separate battery pack for the motor driver and connect all board grounds together (common GND).

📌 Pin Mapping

Arduino UNO (`esp_arduino_obstacle.ino`)

Pin	Function
`2` (Rx)	SoftwareSerial — receive from ESP8266
`3` (Tx)	SoftwareSerial — transmit to ESP8266
`4`	Right Motor — Backward
`5`	Right Motor — Forward
`6`	Left Motor — Backward
`7`	Left Motor — Forward
`10`	Servo Signal

Pin	Function
`12`	Rear status / back LED
`A1`	Ultrasonic — Trigger
`A2`	Ultrasonic — Echo

ESP32 Client (`esp32_as_client.ino`)

Setting	Value
Wi-Fi SSID	`No_Network`
Wi-Fi Password	`rifat2001005`
Server IP	`192.168.4.1`
Server Port	`80`

ESP32-CAM (`ESP32 cam`)

Setting	Value
Wi-Fi Mode	Station (STA)
Wi-Fi SSID	`R`
Wi-Fi Password	`123456789`
Camera Pinout	AiThinker layout

📁 Repository Structure

```
AI-BASED-HUMAN-FOLLOWING-ROBOT/
│
├── Arduino Code/
│   └── esp_arduino_obstacle.ino     # Motion controller: motors, servo, ultrasonic
│
```

```
├── ESP32 Code/
│   └── esp32_as_client.ino          # Wi-Fi client forwarding angle,distance
│
├── ESP32 Cam module/
│   └── ESP32 cam                    # Camera HTTP server (lo/mid/hi JPEG endpoints)
│
├── ESP8266/
│   └── esp8266_as_server.ino        # Wi-Fi AP server, relays data to Arduino
│
├── Images/
│   └── (project photos)
│
└── README.md
```

🚀 Setup & Flashing Guide

1. Flash the Arduino

1. Open **Arduino IDE** and select board: `Arduino UNO`.
2. Open `Arduino Code/esp_arduino_obstacle.ino`.
3. Connect Arduino via USB, select the correct COM port.
4. Click **Upload**.

2. Flash the ESP8266 Server

1. Install the **ESP8266 board package** in Arduino IDE (via Board Manager).
2. Open `ESP8266/esp8266_as_server.ino`.
3. Select board: `Generic ESP8266 Module`.
4. Upload. The ESP8266 will create Wi-Fi AP `No_Network` and listen on port 80.

3. Flash the ESP32 Client

1. Install the **ESP32 board package** in Arduino IDE.
2. Open `ESP32 Code/esp32_as_client.ino`.
3. Update Wi-Fi credentials if needed.
4. Select board: `ESP32 Dev Module` and upload.
5. Open **Serial Monitor at 115200 baud** to send test messages.

4. Flash the ESP32-CAM *(Optional)*

1. Select board: `AI Thinker ESP32-CAM`.

2. Open `ESP32 Cam module/ESP32 cam`.
3. Set your local Wi-Fi SSID/password.
4. Pull IO0 to GND during upload; release after flashing completes.
5. Open Serial Monitor — note the printed IP address.
6. Visit `http://<ip>/cam-mid.jpg` in a browser to verify the feed.

5. Wiring Diagram (Logical)

```
PC / Vision Host
      │  USB Serial
      ▼
  ESP32 Client ─── Wi-Fi ──▶ ESP8266 AP Server
                                  │ TX → Arduino pin 2
                                  ▼
                            Arduino UNO
                            ├── L298N ──▶ Left Motor
                            ├── L298N ──▶ Right Motor
                            ├── Servo  (pin 10)
                            └── HC-SR04 (A1 trig / A2 echo)
```

🧠 Control Logic (Deep Dive)

The Arduino reads each `angle,distance` message and responds based on three angular zones:

Zone 1 — Centered (angle 85°–95°)

Servo is set to **90°**. Motion is decided purely by distance:

Distance	Action
`145 - 155`	**STOP** — target at ideal distance
`> 155`	**MOVE FORWARD** — duration scales with `distance - 155`
`< 145`	**MOVE BACKWARD** — only if rear sensor reads > 30 cm

Zone 2 — Target Left (angle > 95°)

1. Servo rotates to the angle value.
2. Robot **turns left** — opposite motor directions, delay proportional to `angle - 90`.

3. After turning, forward/stop/backward distance logic is applied.

Zone 3 — Target Right (angle < 85°)

1. Servo rotates to the angle value.
2. Robot **turns right** — symmetric logic to Zone 2.
3. After turning, forward/stop/backward distance logic is applied.

Rear Obstacle Guard

Before any backward movement, the Arduino samples the HC-SR04:

```
If rear_distance ≤ 30 cm → CANCEL backward move
```

This prevents reversing into walls or objects behind the robot.

🧪 Test Inputs & Examples

Type these into the **ESP32 Serial Monitor** (115200 baud) to test without a vision system:

Input String	Expected Behavior
`90,160`	Servo centers → robot moves **forward**
`90,150`	Servo centers → robot **stops**
`90,130`	Servo centers → robot moves **backward** (if rear clear)
`60,140`	Target right → robot turns right, then applies distance logic
`120,160`	Target left → robot turns left, then moves forward

> **Note:** Each message must end with `\n`. The Arduino reads with `readStringUntil('\n')`.

🎛️ Calibration & Tuning

Parameter	Default	Notes
Ideal follow distance (min)	`145`	Edit constant in Arduino sketch
Ideal follow distance (max)	`155`	Edit constant in Arduino sketch
Rear obstacle threshold	`30 cm`	Modify ultrasonic guard constant
Turn delay scaling	Linear	Replace with PWM/PID for smoother motion
Motor speed	Full (digital)	Add `analogWrite()` for variable PWM speed
Servo center	`90°`	Adjust initial `servo.write()` value

👁️ Adding a Vision/Detection Module

This repo provides the **robot side** only. To make it fully autonomous, you need something that outputs `angle,distance\n` to the ESP32's Serial port.

Option A — OpenCV on a PC

```
import cv2, serial, numpy as np
from urllib.request import urlopen

cam_url = "http://<ESP32-CAM-IP>/cam-mid.jpg"
ser = serial.Serial('/dev/ttyUSB0', 115200)  # ESP32 client port

while True:
    img_arr = np.frombuffer(urlopen(cam_url).read(), dtype=np.uint8)
    frame = cv2.imdecode(img_arr, cv2.IMREAD_COLOR)

    # --- Your detection logic here ---
    # e.g., HOG detector, YOLO, MediaPipe Pose, etc.
    angle, distance = detect_person(frame)
    # ---------------------------------

    ser.write(f"{angle},{distance}\n".encode())
```

Option B — Dedicated Vision Module

Any device (Raspberry Pi, Jetson Nano, etc.) running person detection that outputs `angle,distance\n` over USB serial to the ESP32 will work with no firmware changes.

Option C — Manual Testing

Type values directly into the ESP32 Serial Monitor — ideal for debugging motor and servo responses without any vision hardware.

🛠️ Troubleshooting

Problem	Likely Cause	Fix
Robot doesn't respond to inputs	ESP8266/ESP32 not connected or wrong IP	Verify Wi-Fi; confirm server IP is `192.168.4.1`
Motors don't spin	L298N wiring or enable pin not set	Check ENA/ENB; verify separate motor power supply
Servo not moving	Wrong pin or insufficient power	Confirm servo on pin 10; check servo VCC
Robot turns but doesn't drive	Motor driver logic mismatch	Verify IN1–IN4 wiring matches sketch pins
Arduino blocked / silent	SoftwareSerial conflict	Don't use pins 2/3 for debug Serial simultaneously
ESP32-CAM no video	Wrong camera pins or PSRAM off	Select AiThinker layout; enable PSRAM in board settings
Backward motion never triggered	Rear obstacle within 30 cm	Clear path behind robot or raise threshold

---

🔐 Security Notes

> ⚠️ This is an **experimental / demo project**. Before any open-environment deployment:

- **Default credentials are hardcoded** in all sketches — change them.
- The `angle,distance` channel has **no authentication or encryption**.
- The ESP32-CAM stream is an **open HTTP server** accessible to anyone on the network.

📜 License

This project does not currently include a license file. We recommend adding the MIT License.

🙏 Acknowledgements

Team Robo Inquisitives — Department of Computer Science & Engineering, 2023.

Contributors:

- @rifat87
- @Numan2001011
- @jalish-bdu
- @saifulshark

📺 Full demo and walk-through: **YouTube Tutorial**

WATERMARK WATERMARK WATERMARK